

INTRODUCTION TO INHERITANCE

Object-Oriented Programming — Student Manual

Course length: 7.5 hours | 3 classes × 2.5 hours

Language used in examples: Java

What you will learn

- The concept of inheritance
- How to extend classes
- How to override and access superclass methods
- How to call constructors during inheritance
- How to apply information hiding
- Which methods you cannot override

Disclaimer:

The following manual, book and AI tools were consulted in compiling this document:

- PROG6112 Module Outline
- Java Programming, 10th Edition, by Joyce Farrell
- Claude and ChatGPT.

Compiled by **VW Kambale**

Table of Contents

This manual is organized into three classes of 2.5 hours each, for a total of 7.5 hours of instruction.

1. How to Use This Manual
2. Class 1 (2.5h) — The Concept of Inheritance & Extending Classes
3. Class 2 (2.5h) — Overriding, Accessing Superclass Methods & Constructors
4. Class 3 (2.5h) — Information Hiding & Methods You Cannot Override
5. Final Review & Comprehensive Exercise
6. Glossary of Key Terms
7. Answer Guidance for Instructors

How to Use This Manual

This manual is your companion for the three classes of the Introduction to Inheritance course. Each class follows the same structure, so you always know what to expect:

- Learning Objectives — what you should be able to do by the end of the class.
- Definitions — precise, boxed definitions of key terms.
- Explanations — plain-language descriptions of each concept.
- Example Code — runnable Java examples that illustrate the concept.
- Exercises — hands-on practice, to be completed during or after class.

Work through each section in order. Type out and run every example yourself — do not just read the code. Attempt every exercise before checking the answer guidance at the end of the manual.

DEFINITION — Inheritance

A mechanism in object-oriented programming that allows a new class (the subclass) to acquire the fields and methods of an existing class (the superclass), enabling code reuse and the modelling of "is-a" relationships between classes.

Class 1 — The Concept of Inheritance & Extending Classes

 Duration: 2.5 hours

Learning Objectives

- ☐ Explain what inheritance is and why it is used
- ☐ Identify superclass and subclass relationships
- ☐ Extend a class using the correct syntax
- ☐ Override a superclass method
- ☐ Access a superclass's method from within a subclass

1.1 The Concept of Inheritance

DEFINITION — Superclass (Parent Class)

The class whose fields and methods are inherited by another class. Also called the base class or parent class.

DEFINITION — Subclass (Child Class)

A class that inherits fields and methods from a superclass and may add its own fields, methods, or override inherited ones. Also called the derived class or child class.

Inheritance models an "is-a" relationship. For example, a Car is a Vehicle, and a Dog is an Animal. Instead of writing the shared behaviour of Vehicle again inside Car, Car simply inherits it. This avoids duplicated code, keeps related classes organized in a hierarchy, and makes programs easier to extend and maintain.

Why use inheritance?

- Code reuse — common fields and methods are written once, in the superclass.
- Logical organization — classes are arranged in a hierarchy that mirrors real-world relationships.
- Extensibility — new subclasses can be added without modifying existing code.
- Polymorphism (covered in a later course) relies on inheritance as its foundation.

Example — a simple hierarchy

Consider a school system. Every Person has a name and an age. A Student is a Person who also has a student number. A Teacher is a Person who also has a subject. Both Student and Teacher are-a Person, so both can inherit from a Person superclass.

1.2 How to Extend Classes

DEFINITION — extends keyword

In Java, the keyword used in a class declaration to indicate that the new class (subclass) inherits from an existing class (superclass). Syntax: `class SubclassName extends SuperclassName { }`

To create a subclass in Java, you use the extends keyword after the subclass name. The subclass automatically gains access to the public and protected fields and methods of the superclass.

```
// Superclass
public class Person {
    protected String name;
```

```

    protected int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void introduce() {
        System.out.println("Hi, my name is " + name + " and I am " + age + " years
old.");
    }
}

// Subclass
public class Student extends Person {
    private String studentNumber;

    public Student(String name, int age, String studentNumber) {
        super(name, age); // calls the Person constructor
        this.studentNumber = studentNumber;
    }
}

```

Notice that Student did not need to redeclare the name or age fields, or the introduce() method — it inherited them automatically from Person.

Key rules when extending classes

- Java supports single inheritance for classes: a class can extend only one superclass.
- Every class in Java implicitly extends Object if no other superclass is specified.
- A subclass inherits all public and protected members, but not private members, of its superclass.
- Constructors are not inherited — each class must define its own (see Class 2).

1.3 Overriding and Accessing Superclass Methods

DEFINITION — Method Overriding

Providing a new implementation, in a subclass, of a method that is already defined in its superclass. The overriding method must have the same name, parameter list, and return type (or a covariant return type) as the superclass method.

DEFINITION — @Override annotation

An optional but recommended Java annotation placed above a method to indicate it is intended to override a superclass method. The compiler will raise an error if no matching superclass method exists, helping catch mistakes early.

Overriding lets a subclass provide its own, more specific version of a behaviour inherited from the superclass.

```

public class Person {
    public void introduce() {
        System.out.println("I am a person.");
    }
}

public class Student extends Person {

```

```
@Override
public void introduce() {
    System.out.println("I am a student.");
}
}
```

DEFINITION — super keyword (method access)

A Java keyword used inside a subclass to explicitly refer to a member (method or field) of its superclass, most commonly to call the superclass version of an overridden method.

Sometimes you want to extend, rather than completely replace, the superclass behaviour. Use `super.methodName()` to call the superclass's version of a method from inside the overriding method.

```
public class Student extends Person {
    @Override
    public void introduce() {
        super.introduce();           // run the Person version first
        System.out.println("...and I am also a student.");
    }
}
```

Overriding vs. Overloading

Do not confuse overriding with overloading. Overriding replaces a method in a subclass using the exact same signature. Overloading defines multiple methods with the same name but different parameter lists, usually within the same class.

Class 1 — Exercises**Exercise 1 — Identify the relationship**

For each pair below, state whether an "is-a" (inheritance) relationship makes sense, and identify which class would be the superclass and which the subclass:

- Animal / Cat
- Book / Library
- Shape / Circle
- Engine / Car
- Employee / Manager

Exercise 2 — Create a class hierarchy

Create a superclass named `Vehicle` with fields `brand` (`String`) and `topSpeed` (`int`), a constructor, and a method `describe()` that prints the brand and top speed. Then create a subclass named `Car` that extends `Vehicle` and adds a field `numberOfDoors` (`int`).

Exercise 3 — Override a method

In your Car class from Exercise 2, override `describe()` so that it prints the same information as Vehicle's version, plus the number of doors. Use `super.describe()` inside your override rather than duplicating the print statement.

Class 2 — Constructors and Information Hiding

 Duration: 2.5 hours

Learning Objectives

- ☐ Explain how constructors behave in an inheritance hierarchy
- ☐ Call a superclass constructor explicitly using `super()`
- ☐ Explain the purpose of information hiding
- ☐ Apply access modifiers correctly to enforce information hiding
- ☐ Use encapsulation together with inheritance

2.1 Recap: Class 1

Before continuing, make sure you are comfortable with: the concept of inheritance, the `extends` keyword, method overriding, and the use of `super.methodName()` to call a superclass method. If any of these feel unclear, revisit Class 1 before proceeding.

2.2 Calling Constructors During Inheritance

DEFINITION — Constructor Chaining

The process by which a subclass constructor invokes a constructor of its superclass — either explicitly using `super(...)` or implicitly by the compiler — before executing its own body, ensuring the inherited part of the object is fully initialized first.

Constructors are never inherited. Every class must define its own constructor(s). However, when a subclass object is created, the superclass's constructor always runs first, to make sure the inherited fields are properly initialized before the subclass adds its own.

The `super(...)` call

If a subclass constructor does not explicitly call `super(...)`, Java automatically inserts a call to the superclass's no-argument constructor as the first statement. If the superclass has no no-argument constructor, the subclass must call `super(...)` explicitly with matching arguments, and this call must be the first statement in the constructor.

```
public class Person {
    protected String name;
    protected int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Person constructor running");
    }
}

public class Student extends Person {
    private String studentNumber;

    public Student(String name, int age, String studentNumber) {
        super(name, age);           // MUST be the first statement
    }
}
```

```

        this.studentNumber = studentNumber;
        System.out.println("Student constructor running");
    }
}

// new Student("Ana", 20, "S1023") prints, in order:
// Person constructor running
// Student constructor running

```

Rules to remember

- `super(...)`, if used, must be the first line of the constructor.
- You cannot call `super(...)` and `this(...)` in the same constructor.
- Constructor execution always runs from the top of the hierarchy downward: superclass first, then subclass.
- If the superclass has only parameterized constructors, the subclass **MUST** call one explicitly, or a compile-time error occurs.

2.3 Applying Information Hiding

DEFINITION — Information Hiding

A design principle in which the internal data and implementation details of a class are hidden from other classes, and access is only permitted through a controlled, well-defined interface (typically public methods).

DEFINITION — Encapsulation

The technique of bundling an object's data (fields) with the methods that operate on that data, and restricting direct access to the internal fields, most commonly by declaring fields private and exposing getters/setters or other public methods.

Information hiding protects an object's internal state from being changed in unexpected or invalid ways by outside code. It also allows the internal implementation to change later without breaking code that depends on the class.

Access modifiers in Java

Modifier	Same Class	Same Package	Subclass (other pkg)	World
private	Yes	No	No	No
default (none)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

A common and recommended pattern: declare fields private, and provide public getter and setter methods to control how they are read and modified.

```

public class Person {
    private String name;      // hidden – cannot be accessed directly from outside
    private int age;

    public Person(String name, int age) {
        this.name = name;
        setAge(age);          // validation happens in the setter
    }
}

```

```

    }

    public String getName() {
        return name;
    }

    public void setAge(int age) {
        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative");
        }
        this.age = age;
    }
}

```

Notice why `protected` is often used instead of `private` for fields that a subclass needs direct access to: `private` fields are hidden even from subclasses, while `protected` fields remain accessible within the same package and to subclasses anywhere, but still stay hidden from unrelated classes.

Class 2 — Exercises

Exercise 1 — Trace constructor execution

Given a three-level hierarchy `Animal` → `Mammal` → `Dog`, each with its own constructor that prints a message and calls `super(...)`, write down the exact order in which the constructor messages will print when you run `new Dog(...)`.

Exercise 2 — Fix the compile error

The `Employee` class below has only a constructor that takes `(String name, double salary)`. A subclass `Manager` tries to compile with no constructor at all. Explain why this fails to compile, and write the missing `Manager` constructor correctly, calling `super(...)`.

```

public class Employee {
    public Employee(String name, double salary) { ... }
}

public class Manager extends Employee {
    // no constructor written
}

```

Exercise 3 — Apply information hiding

Take your `Vehicle/Car` classes from Class 1. Make all fields `private`. Add public getters for every field, and a public setter for `topSpeed` that rejects negative values by throwing an `IllegalArgumentException`. Update your constructors to use the setter for validation.

Class 3 — Methods You Cannot Override

 Duration: 2.5 hours

Learning Objectives

- ☐ Identify which methods cannot be overridden in Java, and why
- ☐ Use the final keyword to prevent a method from being overridden
- ☐ Explain the relationship between private, static, and final methods and overriding
- ☐ Apply everything learned so far in a combined, multi-class exercise

3.1 Recap: Classes 1 and 2

Before this final class, you should be comfortable with: extending classes, overriding methods, using super for both methods and constructors, constructor chaining, and applying information hiding through access modifiers.

3.2 Methods That Cannot Be Overridden

Not every method inherited from a superclass can be overridden. Java prevents overriding in three specific situations:

1. final methods

DEFINITION — final method

A method declared with the final keyword, which prevents any subclass from overriding it. Attempting to override a final method causes a compile-time error.

```
public class Person {
    public final void showId() {
        System.out.println("ID logic that must never change");
    }
}

public class Student extends Person {
    // The following would NOT compile:
    // public void showId() { ... } // Error: cannot override final method
}
```

Developers mark a method final when its behaviour is critical and must remain identical across every subclass — for example, a security check or a core calculation that should never be altered.

2. static methods

DEFINITION — Method Hiding

What happens when a subclass declares a static method with the same signature as a static method in its superclass. This is not overriding — it is hiding, because static methods belong to the class, not to object instances, and are resolved at compile time based on the reference type.

static methods belong to the class itself, not to individual objects, so they cannot be overridden. A subclass can declare a static method with the same signature, but this hides the superclass version rather than overriding it — the method that runs is chosen based on the declared (compile-time) type of the reference, not the actual object type.

```

public class Person {
    public static void category() {
        System.out.println("Category: Person");
    }
}

public class Student extends Person {
    public static void category() { // hides, does not override
        System.out.println("Category: Student");
    }
}

```

3. private methods

private methods are not inherited by subclasses at all, so they cannot be overridden — they are not even visible outside the class that declares them. If a subclass declares a method with the same name and signature as a private method in its superclass, it is treated as a brand-new, unrelated method, not an override.

Constructors are not overridden either

Although not "methods" in the strict sense, it is worth restating: constructors are never inherited and therefore can never be overridden. Each class defines its own constructors, optionally chained to the superclass via `super(...)`, as covered in Class 2.

3.3 Summary Table — Can It Be Overridden?

Method type	Can it be overridden?	Reason
public / protected instance method	Yes	This is the standard case that overriding is designed for.
final method	No	Explicitly locked by the developer to prevent any change in behaviour.
static method	No (can be hidden)	Belongs to the class, resolved at compile time, not polymorphic.
private method	No	Not inherited — invisible to subclasses entirely.
Constructor	No (not applicable)	Constructors are never inherited, so "override" does not apply.

Class 3 — Exercises

Exercise 1 — Spot the error

The code below fails to compile. Identify the exact line that causes the error and explain why.

```

public class Shape {
    public final double area() {
        return 0.0;
    }
}

```

```
public class Circle extends Shape {  
    private double radius;  
  
    public double area() {           // <-- line in question  
        return Math.PI * radius * radius;  
    }  
}
```

Exercise 2 — Override vs. hide vs. neither

For each pair of methods below (one in a superclass, one in a subclass with the same name and signature), state whether the subclass method overrides, hides, or is unrelated to the superclass method:

- public void run() in superclass; public void run() in subclass
- private void log() in superclass; private void log() in subclass
- public static void init() in superclass; public static void init() in subclass
- public final void save() in superclass; public void save() in subclass

Exercise 3 — Comprehensive challenge

Design a three-class hierarchy: Shape (superclass), and Rectangle and Triangle (subclasses). Requirements:

- Shape has a private field color, a protected constructor that sets it, a public getColor(), and a public method describe() that prints the color.
- Shape also has a public final method printHeader() that prints "--- Shape Report ---" and cannot be changed by any subclass.
- Rectangle and Triangle each add their own dimension fields, call super(color) in their constructors, and override describe() to call super.describe() and then print their own area.
- Write a small main() method that creates one Rectangle and one Triangle and calls printHeader() and describe() on each.

Final Review & Comprehensive Exercise

 *Suggested duration: last 30–40 minutes of Class 3*

Use this section to consolidate everything covered across the three classes.

Concept Checklist

Learning Objectives

- ☐ I can explain what inheritance is and describe an is-a relationship
- ☐ I can create a subclass using `extends`
- ☐ I can override a superclass method and use `@Override` correctly
- ☐ I can call a superclass method from a subclass using `super.method()`
- ☐ I can call a superclass constructor from a subclass using `super(...)`
- ☐ I can explain why constructors are never inherited
- ☐ I can apply information hiding using `private`, `protected`, and `public`
- ☐ I can list the three cases where a method cannot be overridden

Comprehensive Exercise — Library System

Exercise Final — Build a small library hierarchy

Create a superclass `LibraryItem` with private fields `title` (`String`) and `isCheckedOut` (`boolean`, default `false`), a constructor, a public `getTitle()`, a public final method `printBarcode()` that prints a fixed placeholder barcode format, and a public method `checkOut()` that sets `isCheckedOut` to `true` and prints a message.

Create two subclasses, `Book` (adds an `author` field) and `DVD` (adds a `runtimeMinutes` field). Each subclass constructor must call `super(title)` correctly. Each subclass should override `checkOut()` to call `super.checkOut()` and then print an item-specific message.

Write a `main()` method that creates one `Book` and one `DVD`, checks both out, and calls `printBarcode()` on each. Confirm for yourself that `printBarcode()` cannot be overridden — try it and observe the compiler error, then remove the attempt.

Glossary of Key Terms

Inheritance — A mechanism allowing a class to acquire fields and methods from another class.

Superclass — The class being inherited from (parent/base class).

Subclass — The class that inherits from another class (child/derived class).

extends — The Java keyword used to declare that one class inherits from another.

Overriding — Redefining, in a subclass, a method already defined in its superclass, using the same signature.

@Override — An annotation that tells the compiler a method is intended to override a superclass method.

super — A keyword used to access a superclass's methods, fields, or constructor from a subclass.

Constructor chaining — The process of a subclass constructor invoking its superclass constructor before running its own body.

Information hiding — Restricting access to an object's internal data so it can only be reached through a controlled interface.

Encapsulation — Bundling data and the methods that operate on it, typically hiding the data behind private fields.

Access modifier — A keyword (private, default, protected, public) that controls the visibility of a class member.

final method — A method that cannot be overridden by any subclass.

static method — A class-level method that cannot be overridden, only hidden.

Method hiding — When a subclass static method has the same signature as a superclass static method; resolved at compile time.

Answer Guidance for Instructors

This section provides brief guidance for facilitators reviewing student answers. It is not intended to be read by students before attempting the exercises.

Class 1

- Exercise 1: Animal/Cat, Shape/Circle, and Employee/Manager are valid is-a relationships. Book/Library and Engine/Car are has-a (composition), not inheritance.
- Exercises 2–3: Check that students used extends, called methods correctly, and used `super.describe()` rather than duplicating logic.

Class 2

- Exercise 1: Expected order — Animal constructor, then Mammal constructor, then Dog constructor (top of hierarchy first).
- Exercise 2: Fails because Employee has no no-argument constructor, so the compiler cannot insert an implicit `super()` call in Manager; students must add an explicit constructor calling `super(name, salary)`.
- Exercise 3: Confirm all fields are private, getters/setters are public, and validation logic lives in the setter.

Class 3

- Exercise 1: The `area()` method in Circle causes a compile error because `Shape.area()` is declared final and cannot be overridden.
- Exercise 2: `run()` overrides; `log()` is unrelated (private, not inherited); `init()` hides (static); `save()` — subclass cannot declare it at all if truly matching final signature, so this should be flagged as a compile error, reinforcing that final applies per-method regardless of static/instance.
- Exercise 3 and Final Exercise: Check students correctly use `super(color)/super(title)` in constructors, override `describe()/checkOut()` using super calls, and confirm they observe (and then remove) the compiler error when attempting to override the final method.